ara aclarar un concepto: en Git, **"clonar" significa descargar un repositorio entero desde cero a una carpeta nueva**. Lo que tú quieres hacer se llama **"fusionar" (merge) o "traer" (pull)** los cambios de la rama `desarrollo` hacia tu rama nueva.

Imagina que la rama `desarrollo` es un documento que otra persona actualizó. Tú no quieres "clonarlo" (hacer una copia nueva), sino **traer esas actualizaciones y mezclarlas con tus propios cambios** en tu rama actual.

Aquí tienes el paso a paso exacto (con comandos de Git Bash o la terminal de GitHub) para hacerlo sin perder tu trabajo:

⚠ Paso 0: Asegura tu trabajo

Antes de mezclar cosas, asegúrate de que tus cambios actuales estén guardados en un "commit". Si no lo has hecho, hazlo ahora:

```bash
git add .
git commit -m "feat: mis cambios antes de traer desarrollo"
```

🔍 Paso 1: Verifica que estás en TU rama

Asegúrate de no estar en `main` ni en `desarrollo`, sino en tu rama nueva:

```bash
git branch
```

(Debe aparecer un asterisco `*` al lado de tu rama nueva).

⬇ Paso 2: Descargar lo último de `desarrollo`

Le pedimos a Git que vaya al servidor y traiga los cambios de la rama `desarrollo`:

```bash
```

```
git fetch origin
```

🔀 Paso 3: Fusionar `desarrollo` con tu rama

Ahora traemos los cambios de la rama remota `desarrollo` y los metemos en tu rama local:

```
bash

git merge origin/desarrollo
```

(También puedes usar `git pull origin desarrollo`, que hace el `fetch` y el `merge` en un solo paso, pero hacerlo separado es más seguro).

🚨 Paso 4: Resolver conflictos (Si los hay)

Si tus compañeros modificaron las **mismas líneas de código** que tú en los mismos archivos, Git se detendrá y dirá `CONFLICT (content)`.

Si eso pasa:

1.Abre los archivos en conflicto (los verás con `git status`).
2.Busca las marcas `<<<<<<<`, `=======`, `>>>>>>>`. Elige qué código dejar (el tuyo, el de ellos, o una mezcla de los dos).
3.Guarda el archivo.
4.Dile a Git que ya lo resolviste:

```
bash

git add archivo_resuelto.sql
git commit -m "merge: resolviendo conflictos con desarrollo"
```

🚀 Paso 5: Subir tu rama actualizada

Ahora que tu rama nueva tiene tus cambios **Y** los cambios de `desarrollo`, súbela al servidor:

```
bash

git push origin nombre-de-tu-rama-nueva
```

💡 Resumen visual de lo que acaba de pasar:

Antes del merge:

•**Rama desarrollo (servidor):** `A -> B -> C`
•**Tu rama (local):** `A -> D (tus cambios)`

Después del merge (Paso 3):

•**Tu rama (local):** `A -> B -> C -> D (tus cambios)`

¡Listo! Ahora tu rama tiene lo último que había en `desarrollo` mezclado con tus archivos editados.

Para clonar un repositorio y que se descargue directamente en la rama `desarrollo` (en lugar de `main`), solo debes agregar el parámetro `-b` (de branch) seguido del nombre de la rama.

Aquí tienes los comandos exactos:

🎯 Opción 1: Clonar todo el repositorio, pero posicionado en `desarrollo` (Recomendado)

Esta es la forma estándar. Descarga todo el historial del repositorio, pero tu carpeta de trabajo se abrirá directamente en la rama `desarrollo`.

1.Abre **Git Bash** o **CMD**.
2.Ve a la carpeta donde quieres guardar el proyecto:

```bash
cd C:\ruta\donde\quieres\el\proyecto
```

3.Ejecuta el comando de clonación con la bandera `-b`:

```bash
git clone -b desarrollo https://dev.azure.com/tu-empresa/tu-proyecto/_git/tu-repositorio
```

(Recuerda reemplazar la URL ficticia por la URL real de tu repositorio en Azure).
4.Entra a la carpeta que se acaba de crear:

```bash
cd tu-repositorio
```

5.Verifica que estás en la rama correcta:

```bash
git branch
```

Lo que verás:

```text
* desarrollo
  remotes/origin/HEAD -> origin/main
  remotes/origin/desarrollo
  remotes/origin/main
```

(El asterisco `*` debe estar en `desarrollo`).

🚀 Opción 2: Clonar SOLO la rama `desarrollo` (Ahorra espacio y tiempo)

Si el repositorio es muy grande y **solo** te interesa esa rama (no quieres descargar el historial de `main` ni de otras ramas), usa el parámetro `--single-branch`.

```bash
git clone -b desarrollo --single-branch https://dev.azure.com/tu-empresa/tu-proyecto/_git/tu-repositorio
```

Diferencia clave:

•Con la **Opción 1**, si luego haces `git checkout main`, funcionará de inmediato porque ya está descargada.
•Con la **Opción 2**, si haces `git checkout main`, Git te dirá que la rama no existe localmente, pero la puedes traer con `git fetch origin main:main`.

💡 Nota sobre el mensaje anterior

En tu pregunta anterior mencionaste que ya tenías una rama creada y querías traer los cambios de `desarrollo`. Si el objetivo de **clonar desde cero** es porque el proyecto se dañó o quieres empezar de nuevo sin perder lo que tenías en tu otra carpeta, entonces estos comandos son los correctos.

Si lo que querías era **actualizar tu carpeta actual** (la que ya tiene tu rama creada) con los cambios de `desarrollo`, no necesitas volver a clonar, solo usa:

```bash
git fetch origin
git merge origin/desarrollo
```